

Dossier Projet PC3R

Jean-Charles KAING 3808150

Shiyao CHEN 28707756

Mai 2024



Lien vers le répertoire git : <https://stl.algo-prog.info/3808150/projet-PC3R.git>

Contents

1	Introduction	3
1.1	Description de l'application	3
1.2	Description de l'API Web	3
1.3	Descriptions des fonctionnalités de l'application	3
1.4	Cas d'utilisations classiques	3
1.5	Liste des données	3
1.6	Mises à jour des données et appel à l'API externe	4
2	Serveur	4
2.1	Choix de l'approche	4
2.2	Composition et fonctionnalités associées	4
2.2.1	Code de ServletUsers	4
3	Client	12
3.1	Plan du site	12

1 Introduction

1.1 Description de l'application

Le sujet de l'application est un site de critiques et d'évaluations de films. L'application permet aussi à un utilisateur d'envoyer des posts visibles par tous les autres. Aussi, nous avons penser à donner la possibilité aux utilisateurs de créer des événements afin de se retrouver et regarder un film, ce ne sera pas le cas.

1.2 Description de l'API Web

L'API choisie est l'api The Movie DataBase (TMBD), donc la description est disponible à l'adresse suivante : <https://developer.themoviedb.org/reference/intro/getting-started>. Elle propose des films et des séries, ainsi que les informations liées à ces films et séries. L'application utilisera ces contenus.

1.3 Descriptions des fonctionnalités de l'application

L'application permettra de noter des films. Aussi, l'application permettra de voir quel film sont les populaires en ce moment.

1.4 Cas d'utilisations classiques

Bob s'inscrit sur l'application. Il consulte ensuite une fiche de film en cliquant dessus, le note et le commente. Puis il se déconnecte.

Alice se connecte. Elle consulte ensuite une fiche de film, puis lit la note et le commentaire posté par Bob. Elle poste ensuite un message.

Bob se connecte et consulte le message qu'Alice a posté. Il poste un autre message. Il se déconnecte.

1.5 Liste des données

- Informations concernant les utilisateurs tels que l'identifiant, le mot de passe, le mail et la date de naissance.
- Commentaires des utilisateurs
- Notes des films
- Posts des utilisateurs
- Informations sur les fiches de films
- Informations sur les fiches des acteurs

1.6 Mises à jour des données et appel à l'API externe

Les données sont mises à jour après chaque post d'utilisateur, à chaque fois qu'un utilisateur va sur la page principale et à chaque fois qu'un nouveau film est consulté. L'API externe est appelé à chaque requête concernant une donnée de film.

2 Serveur

2.1 Choix de l'approche

On utilisera une approche ressource dans le cadre d'un projet dynamique web à l'aide de Tomcat Catalina et de MongoDB Atlas pour stocker les données. Une API REST est créée.

2.2 Composition et fonctionnalités associées

2.2.1 Code de ServletUsers

Pour manipuler les utilisateurs

```
@WebServlet("/Users")
public class ServletUsers extends HttpServlet {
    private static final long serialVersionUID = 1L;

    Gson gson;
    MongoClient mongoClient;
    MongoDB database;

    /**
     * @see HttpServlet#HttpServlet()
     */
    public ServletUsers() {
        super();
    }

    public void init() {
        String uri =

        ↪ "mongodb+srv://dysm1869:PYeYBFWVX677vQgM@cluster0.thzsihz.mongodb.net/?retryWrites=
        mongoClient = MongoClient.create(uri);
        database = mongoClient.getDatabase("BDD");

        // Prépare un objet Gson pour convertir les chaînes en
        ↪ JSON et vice versa
        this.gson = new Gson();
    }
```

```

    public void destroy() {
        mongoClient.close();
    }

    /**
     * @see HttpServlet#doGet(HttpServletRequest request,
     * ↪ HttpServletResponse response)
     */
    protected void doGet(HttpServletRequest request,
        ↪ HttpServletResponse response) throws
        ↪ ServletException, IOException {
        String _id = request.getParameter("_id");

        if (_id!=null) {
            ObjectId id = new ObjectId(_id);

            MongoCollection<Document> collection =
                ↪ database.getCollection("users");

            Document doc = collection.find(new
                ↪ Document("_id", id)).first();

            response.setStatus(HttpServletResponse.SC_OK);
            response.setContentType("application/json");
            response.setCharacterEncoding("UTF-8");
            response.getWriter().write(doc.toJson());
            ↪ // Renvoie les informations du
            ↪ document en tant que réponse HTTP
        }
        else {
            String username =
                ↪ request.getParameter("username");

            if (username!=null)
                ↪ {
                    MongoCollection<Document>
                        ↪ collection =
                        ↪ database.getCollection("users");

                    Document doc =
                        ↪ collection.find(new
                        ↪ Document("username",
                        ↪ username)).first();

                    response.setStatus(HttpServletResponse.SC_OK);

```

```

        response.setContentType("application/json");
        response.setCharacterEncoding("UTF-8");
        response.getWriter().write(doc.toJson());
        ↪ // Renvoie les informations
        ↪ du document en tant que
        ↪ réponse HTTP
    }
    else {
        System.out.println("L'identifiant
        ↪ est null");
        response.sendError(HttpServletResponse.SC_NOT_FOUND,
        ↪ "Document not found");
    }
}

}

/**
 * @see HttpServlet#doPost(HttpServletRequest request,
 * ↪ HttpServletResponse response)
 */
protected void doPost(HttpServletRequest request,
    ↪ HttpServletResponse response) throws
    ↪ ServletException, IOException {
    // créer un utilisateur
    if (username!=null && password!=null &&
    ↪ mail!=null && birthday!=null) {
        Document doc = new Document("username",
        ↪ username)
            .append("password",
            ↪ password)
            .append("mail", mail)
            .append(birthday,
            ↪ birthdate);
        database.getCollection("users").insertOne(doc);

        response.setStatus(HttpServletResponse.SC_OK);
        response.setContentType("application/json");
        response.setCharacterEncoding("UTF-8");
        response.getWriter().write(doc.toJson());
        ↪ // Renvoie les informations du
        ↪ document en tant que réponse HTTP

        ObjectId id = doc.getObjectId("_id");
        System.out.println("L'_id de
        ↪ l'utilisateur inséré est : " +
        ↪ id.toHexString());
    }
}

```

```

    }
    else {
        response.sendError(HttpServletResponse.SC_BAD_REQUEST,
            ↪ "Les données fournies sont
            ↪ incomplètes.");
        System.out.println("Pas de nouveau
            ↪ utilisateur créé");
    }
    */

    // Lit le corps de la requête
    StringBuilder sb = new StringBuilder();
    String line;
    try (BufferedReader reader = request.getReader()) {
        while ((line = reader.readLine()) != null) {
            sb.append(line);
        }
    }

    try {
        String jsonString = sb.toString();

        JSONObject jsonObject2 = new JSONObject(jsonString);
        String username =
            ↪ jsonObject2.getString("username");

        if (database.getCollection("users").find(new
            ↪ Document("username",
            ↪ username)).first() != null) {

            ↪ response.sendError(HttpServletResponse.SC_CONFLI
            ↪ "username déjà utilisé");
            System.out.println("username déjà
                ↪ utilisé");
        }
        else {
            // Convertit le corps de la requête en un
            ↪ objet JSON
            JsonObject jsonObject =
            ↪ gson.fromJson(jsonString,
            ↪ JsonObject.class);

            // Convertit l'objet JSON en un document
            ↪ BSON
            Document doc =
            ↪ Document.parse(gson.toJson(jsonObject));

```

```

        // Insère le document dans la collection

        ↪ database.getCollection("users").insertOne(doc);

doc =
    ↪ database.getCollection("users").find(new
    ↪ Document("username",
    ↪ username)).first();

    ↪ response.setStatus(HttpServletResponse.SC_OK);

    ↪ response.setContentType("application/json");
response.setCharacterEncoding("UTF-8");
response.getWriter().write(doc.toJson());
    ↪ // Renvoie les informations du
    ↪ document en tant que réponse HTTP
    }
} catch (Exception e) {
    // Gère le cas où le corps de la requête n'est pas un
    ↪ JSON valide

    ↪ response.sendError(HttpServletResponse.SC_BAD_REQUEST,
    ↪ "Les données fournies sont incomplètes.");
    System.out.println("Pas de nouveau
    ↪ utilisateur créé");
}

}

/**
 * @see HttpServlet#doPut(HttpServletRequest,
 * ↪ HttpServletResponse)
 */
protected void doPut(HttpServletRequest request,
    ↪ HttpServletResponse response) throws
    ↪ ServletException, IOException {
    // Modifie un utilisateur

    // Lit le corps de la requête
    StringBuilder sb = new StringBuilder();
String line;
    try (BufferedReader reader = request.getReader())
    ↪ {

```



```

        while ((line = reader.readLine()) !=
        ↪ null) {
            sb.append(line);
        }
    }

    try {
        // Convertit le corps de la requête en un
        ↪ objet JSON
        JsonObject jsonObject =
        ↪ gson.fromJson(sb.toString(),
        ↪ JsonObject.class);

        // Récupère l'ID du document à remplacer
        String _id = jsonObject.get("_id").getAsString();

        ObjectId id = new ObjectId(_id);
        String username =
        ↪ jsonObject.get("username").getAsString();
        String oldusername =
        ↪ database.getCollection("users").find(new
        ↪ Document("_id",
        ↪ id)).first().getString("username");
        ↪

        if (!username.equals(oldusername) &&
        ↪ database.getCollection("users").find(new
        ↪ Document("username",
        ↪ username)).first() != null) {

            ↪ response.sendError(HttpServletResponse.SC_CONFLI
            ↪ "username déjà utilisé");
            System.out.println("username déjà
            ↪ utilisé");
        }
        else {
            // Supprime l'ID du document JSON, car il
            ↪ ne doit pas être inclus dans le
            ↪ document de remplacement
            jsonObject.remove("_id");

            // Convertit l'objet JSON en un document
            ↪ BSON
            Document doc =
            ↪ Document.parse(gson.toJson(jsonObject));

```

```

        // Remplace le document dans la
        ↪ collection

        ↪ database.getCollection("users").replaceOne(new
        ↪ Document("_id", id), doc);

    doc =
    ↪ database.getCollection("users").find(new
    ↪ Document("_id", id)).first();

    ↪ response.setStatus(HttpServletResponse.SC_OK);

    ↪ response.setContentType("application/json");
    response.setCharacterEncoding("UTF-8");
    response.getWriter().write(doc.toJson());
    ↪ // Renvoie les informations du
    ↪ document en tant que réponse HTTP
    }
} catch (Exception e) {
    // Gère le cas où le corps de la requête
    ↪ n'est pas un JSON valide
    response.sendError(HttpServletResponse.SC_BAD_REQUEST,
    ↪ "Les données fournies sont
    ↪ incomplètes.");
    System.out.println("Pas de nouveau
    ↪ utilisateur modifié");
}
}

/**
 * @see HttpServlet#doDelete(HttpServletRequest request,
 * ↪ HttpServletResponse response)
 */
protected void doDelete(HttpServletRequest request,
    ↪ HttpServletResponse response) throws
    ↪ ServletException, IOException {
    // permet de supprimer les informations d'un
    ↪ utilisateur
    // L'_id du document à supprimer.
    String _id = request.getParameter("_id");

    if (_id != null) {
        ObjectId id = new ObjectId(_id);

        // Supprimer le document dans MongoDB.

```

```

        DeleteResult result =
        ↪ database.getCollection("users").deleteOne(Filters.eq("_id",
        ↪ id));

        if (result.getDeletedCount() == 0) {
        // Aucun document n'a été supprimé.

        ↪ response.sendError(HttpServletResponse.SC_NOT_FOUND,
        ↪ "Le document à supprimer n'a pas été
        ↪ trouvé.");
    }

    else {
        response.setStatus(HttpServletResponse.SC_OK);
    }
}
else {
    response.sendError(HttpServletResponse.SC_BAD_REQUEST,
    ↪ "Les données fournies sont
    ↪ incomplètes.");
    System.out.println("Pas d'utilisateur
    ↪ supprimé");
}
}
}
}

```

3 Client

Le client utilise JSP.

3.1 Plan du site

Le client dispose d'une page d'accueil où le visiteur peut consulter les films récents et les mieux notés sans être connecter.

Un visiteur peut ensuite s'inscrire ou se connecter en cliquant sur les boutons qui les renvoient vers les formulaire respectives.

Une fois connectée, l'utilisateur peut consulter, depuis la page principale, les fiches de films, les noter et les commenter. Il peut aussi accéder à la liste des posts récents, lui permettant de consuter et d'écrire des posts. Il peut accéder à son profil et le modifier ou le supprimer. Enfin il peut se déconnecter.